Mechanics of Flexible Structures & Soft Robots
MAE-263F
Fall 2026

Homework 2

Due Date: 11:59 PM, 10/19/2026

You should create a single GitHub repository for this class and share it with the instructor (`khalidjm@seas.ucla.edu`). All the homeworks, reports, presentations, and proposal should be uploaded to this repository. Do not create a separate repository for each assignment. Within your repository, create a separate folder for each assignment (e.g., `homework_1`, `homework_2`, `homework_3`, `homework_4`, `homework_5`, `proposal`, `midterm_report`, and `final_report`).

Submission Instructions: This homework is graded entirely through the multiple-choice questions on **Gradescope**. In addition, submit the URL to your GitHub repository on BruinLearn. The `homework_2` folder of your repository should include the following items:

1. **Source code:** One main file named exactly `Homework2.py`, along with any additional files (e.g., functions) as necessary. The main file should require no more than a single command to run.
2. **README file:** A `README.md` file with clear instructions on how to run your code and a short description of each file in your repository.

Your code will not be graded, but it must be in your repository. If no code is submitted, the homework receives a score of 0, regardless of your Gradescope answers.

Introduction: From a Falling Beam to a Loaded Beam

In class, we wrote an implicit simulator for an elastic beam with N nodes falling in a viscous fluid (`main_falling_beam_n_nodes.py`, Section 4.3 of the notes). The beam is a chain of nodes. Neighboring nodes are joined by *stretching springs*, and every interior node carries a *bending spring*. The gradients and Hessians of the elastic energies are provided (`gradEs`, `hessEs`, `gradEb`, `hessEb`). This homework uses that code in two ways.

1. **Part 1: Falling beam (small changes).** You will run `main_falling_beam_n_nodes.py` with a few parameters changed (radii, viscosity, number of nodes) and check the results against simple analytical solutions: buoyant free fall, Stokes' terminal velocity of a sphere, and the terminal velocity of the whole beam.
2. **Part 2: Bending of a simply supported beam (new simulator).** You will turn the same code into a simulator for an aluminum beam that rests on two supports and carries a point load (Section 4.4, Assignment 3 of the notes). You will compare it with Euler–Bernoulli beam theory. The two agree for small loads, but beam theory fails once the deformation is large. The simulation does not have this limitation.

Both parts use the same ingredients: the elastic energies, implicit Euler, and Newton's method. Part 2 adds one new ingredient: *boundary conditions*, handled by solving only for the free degrees of freedom, as in Homework 1.

The questions are designed so that their answers can be checked against analytical solutions (see the appendix). A correct simulator is the easiest way to answer them with confidence.

Part 1: Elastic Beam Falling in a Viscous Fluid

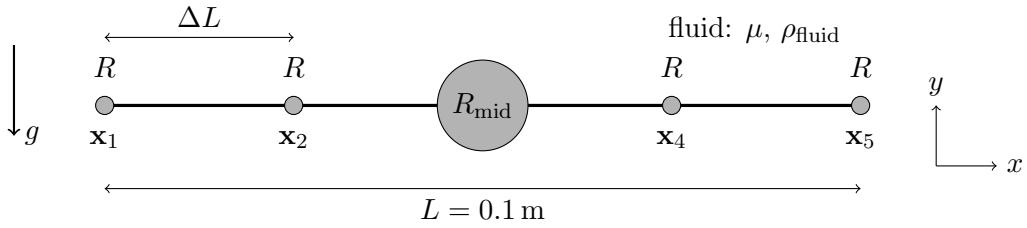


Figure 1: Initial (straight) configuration of the falling beam, shown for $N = 5$. The middle node carries a large sphere of radius R_{mid}; every other node carries a small sphere of radius $R = \Delta L/10$.

Model (Section 4.3 of the notes). A beam of length L is discretized into N nodes (N odd), $\mathbf{x}_k = [x_k, y_k]^T$, with segment length $\Delta L = L/(N-1)$. The degree-of-freedom (DOF) vector is $\mathbf{q} = [x_1, y_1, x_2, y_2, \ldots, x_N, y_N]^T$ (size $2N$). Node k carries a rigid sphere of radius R_k, which sets its mass, weight, and drag:

$$m_k = \tfrac{4}{3}\pi R_k^3 \rho_{\text{metal}}, \qquad W_k = \tfrac{4}{3}\pi R_k^3 \left(\rho_{\text{metal}} - \rho_{\text{fluid}}\right) g \;\; (\text{along } -y), \qquad \text{drag} = -6\pi\mu R_k \, \dot{\mathbf{x}}_k. \quad (1)$$

The weight W_k already includes buoyancy. The beam starts straight along the x-axis and at rest, and then falls under gravity. The y-axis points up, so the beam moves in the $-y$ direction and its vertical velocity is negative.

Default parameters (as in `main_falling_beam_n_nodes.py`):

Quantity	Value
Length, cross-sectional radius	$L = 0.10\,\text{m}$, $r_0 = 10^{-3}\,\text{m}$
Young's modulus	$E = 10^9\,\text{Pa}$ $(EI = E\pi r_0^4/4,\ EA = E\pi r_0^2)$
Densities	$\rho_{\text{metal}} = 7000\,\text{kg/m}^3$, $\rho_{\text{fluid}} = 1000\,\text{kg/m}^3$
Viscosity	$\mu = 1000\,\text{Pa s}$ (`visc` in the code)
Sphere radii	$R_{\text{mid}} = 0.025\,\text{m}$ (middle node), $R = \Delta L/10$ (all other nodes)
Gravity	$g = 9.8\,\text{m/s}^2$
Discretization	$N = 21$, $\Delta t = 10^{-2}\,\text{s}$, total time $50\,\text{s}$

What you need to change. You do **not** need to write a new simulator for Part 1. Start from `main_falling_beam_n_nodes.py` and make it easy to change the following inside `simulate()`:

- the viscosity `visc`;

Copyright notice: Do not copy, distribute, republish, upload, post or transmit anything from the files hosted on the class website without written permission of the instructor

- the sphere radii `R` (for some questions *all* nodes get the same radius);
- the number of nodes `nv` and the total time `totalTime` (already inputs).

One simple way is to add optional arguments:

```
def simulate(nv, dt, totalTime, plotShapes=False, visc=1000.0, R_all=None):
    ...
    R = np.full(nv, deltaL / 10)
    R[midNode] = 0.025
    if R_all is not None:      # same radius at every node
        R[:] = R_all
    ...
```

and delete the line `visc = 1000.0` inside the function. Everything else in the code stays the same.

Running time. Call `simulate(..., plotShapes=False)` when answering the questions; live plotting with `plt.pause` slows the code down a lot. With plotting turned off, a 50 s simulation with $N = 21$ takes about 1–2 minutes, and Question 5 needs four runs, so plan ahead.

Why use equal radii? If every node has the same radius, every node has the same mass, weight, and drag. The beam then falls as a rigid, straight body (no bending), and every node behaves exactly like a single, isolated sphere. This gives simple analytical checks (Appendix A–B). (With $R = 0.025$ m the spheres overlap, since the nodes are only a few millimeters apart. That is fine: the model treats each sphere independently and ignores any interaction between neighboring spheres.)

Part 2: Bending of a Simply Supported Beam

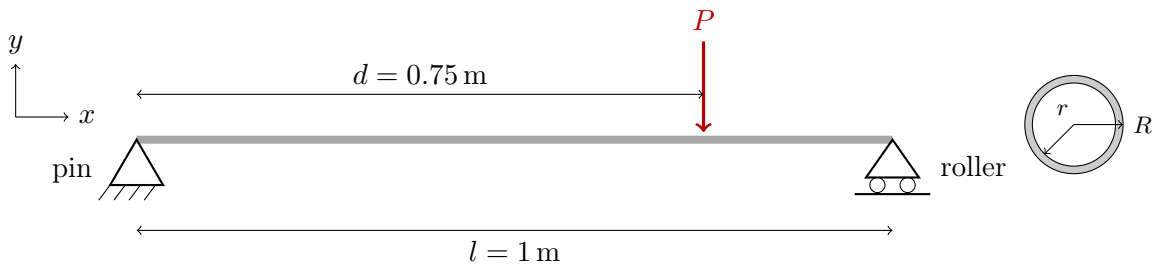


Figure 2: Simply supported aluminum beam with a point load P (left), and its tubular cross section (right). The discrete beam is the same chain of nodes, stretching springs, and bending springs as in Part 1.

Problem (Section 4.4 of the notes, Assignment 3). An aluminum beam of length $l = 1$ m has a circular-tube cross section with outer radius $R = 0.013$ m and inner radius $r = 0.011$ m. Its Young's modulus is $E = 70$ GPa and its density is $\rho = 2700\,\mathrm{kg/m^3}$. The beam is pinned at its left end and rests on a roller at its right end. A downward point load P (along $-y$) is applied at a distance $d = 0.75$ m from the left end. There is no gravity and no fluid.

Copyright notice: Do not copy, distribute, republish, upload, post or transmit anything from the files hosted on the class website without written permission of the instructor

Discrete model. Use $N = 50$ nodes, so $\Delta L = l/(N-1)$. The beam is initially straight, along the x-axis from $(0, 0)$ to $(l, 0)$, and at rest. As in Part 1, the elastic energy is the sum of $N-1$ stretching energies and $N-2$ bending energies, with

$$EI = E\,\frac{\pi}{4}\left(R^4 - r^4\right), \qquad EA = E\,\pi\left(R^2 - r^2\right), \tag{2}$$

and natural curvature zero (the beam is naturally straight). Every DOF has the same lumped mass

$$m = \frac{\pi\left(R^2 - r^2\right) l\rho}{N-1}. \tag{3}$$

The only external force is the load P, applied along $-y$ at the node closest to $x = d$.

Equations of motion. Let $\mathbf{F}^{\text{ext}}$ be the external force vector (size $2N$); its only nonzero entry is $-P$ at the y-DOF of the loaded node. With implicit Euler, the residual and Jacobian are

$$\mathbf{f} = \frac{m}{\Delta t}\left(\frac{\mathbf{q}^{\text{new}} - \mathbf{q}^{\text{old}}}{\Delta t} - \mathbf{u}^{\text{old}}\right) + \left.\frac{\partial E^{\text{elastic}}}{\partial \mathbf{q}}\right|_{\mathbf{q}^{\text{new}}} - \mathbf{F}^{\text{ext}}, \qquad \mathbf{J} = \frac{m}{\Delta t^2}\,\mathbf{I} + \left.\frac{\partial^2 E^{\text{elastic}}}{\partial \mathbf{q}\,\partial \mathbf{q}}\right|_{\mathbf{q}^{\text{new}}}. \tag{4}$$

This is the falling-beam code with the weight $\mathbf{W}$ replaced by $\mathbf{F}^{\text{ext}}$ and with the viscous force removed.

Boundary conditions. The pin fixes both coordinates of the first node, and the roller fixes the y-coordinate of the last node:

$$x_1(t) = 0, \qquad y_1(t) = 0, \qquad y_N(t) = 0. \tag{5}$$

The x-coordinate of the last node is *free*: the roller can slide. As in Homework 1, handle the boundary conditions by assembling the full $\mathbf{f}$ and $\mathbf{J}$ and then solving only for the free DOFs:

```
fixedIndex = np.array([0, 1, 2*N - 1])              # x_1, y_1, y_N (0-based)
freeIndex  = np.setdiff1d(np.arange(2*N), fixedIndex)
...
f_free = f[freeIndex]
J_free = J[np.ix_(freeIndex, freeIndex)]
q_new[freeIndex] -= np.linalg.solve(J_free, f_free)
error = np.linalg.norm(f_free)                       # use the free entries only
```

The fixed DOFs are never updated, so they keep their initial values. The residual at the fixed DOFs is *not* zero (it contains the support reactions), so do not include it in the error.

Summary of changes to `main_falling_beam_n_nodes.py`.

1. New geometry and material: l, R, r, E, ρ, Eq. (2), with $\Delta L = l/(N-1)$.
2. Same mass m (Eq. (3)) for every DOF; no spheres.
3. Replace $\mathbf{W}$ by $\mathbf{F}^{\text{ext}}$; remove the viscous force and its Jacobian.
4. Solve only for the free DOFs, as shown above.
5. Record $y_{\max}(t) = \min_k y_k(t)$, the largest downward displacement of any node at each time step.

The Newton tolerance used in the falling-beam code, `tol` $= 10^{-3}\,EI/l^2$, also works here.

Copyright notice: Do not copy, distribute, republish, upload, post or transmit anything from the files hosted on the class website without written permission of the instructor

Checks. Before running the simulations for the questions, check the following values:

- $\Delta L = 1/49 = 0.020408\,\mathrm{m}$, $EI = 765.29\,\mathrm{N\,m^2}$, $EA = 1.0556 \times 10^7\,\mathrm{N}$, $m = 8.309 \times 10^{-3}\,\mathrm{kg}$.
- The loaded node is node 38 (index 37 in 0-based Python). It sits at $x = 37/49 = 0.7551\,\mathrm{m}$, because no node is exactly at $0.75\,\mathrm{m}$. Its y-DOF is `q[75]`.
- There are $2N - 3 = 97$ free DOFs.
- Newton's method converges within about 10 iterations at every time step. The problem is nonlinear, so the first few steps need several iterations; once the beam has settled, most steps need only one.

Simulation. Simulate $0 \le t \le 1\,\mathrm{s}$ with $\Delta t = 10^{-2}\,\mathrm{s}$ and $N = 50$. The value of $y_{\max}$ at $t = 1\,\mathrm{s}$ is the simulated deflection that you will compare with Euler–Bernoulli theory (Appendix D):

$$y_{\max}^{\mathrm{EB}} = \frac{P\,c\,\left(l^2 - c^2\right)^{1.5}}{9\sqrt{3}\,EI\,l}, \qquad c = \min(d,\ l - d). \tag{6}$$

Each run takes a few seconds. For questions that involve several loads, write a loop over P.

Copyright notice: Do not copy, distribute, republish, upload, post or transmit anything from the files hosted on the class website without written permission of the instructor

Questions (answer on Gradescope)

Answer the following multiple-choice questions on Gradescope using your simulators. Choose the option **closest** to your result. Every question has an analytical answer or an analytical check (see the appendix).

Part 1: Falling beam

Unless a question says otherwise, use the default parameters of Part 1: $N = 21$, $\Delta t = 10^{-2}\,\mathrm{s}$, $\mu = 1000\,\mathrm{Pa\,s}$, $\rho_{\mathrm{metal}} = 7000\,\mathrm{kg/m^3}$, $\rho_{\mathrm{fluid}} = 1000\,\mathrm{kg/m^3}$, $R_{\mathrm{mid}} = 0.025\,\mathrm{m}$, and $R = \Delta L/10$. The y-axis points up, so downward velocities are negative.

1. **Buoyant free fall.** Set the radius of *every* node to 0.025 m and set $\mu = 0$. Simulate for 1 s. What is the vertical velocity of the middle node at $t = 1\,\mathrm{s}$?

   (a) $-11.2\,\mathrm{m/s}$
   (b) $-9.8\,\mathrm{m/s}$
   (c) $-8.4\,\mathrm{m/s}$
   (d) $-1.4\,\mathrm{m/s}$

2. **Terminal velocity of one sphere.** Keep the radius of every node at 0.025 m and restore $\mu = 1000\,\mathrm{Pa\,s}$. Simulate for 1 s. What is the vertical velocity of the middle node at $t = 1\,\mathrm{s}$?

   (a) $-9.53 \times 10^{-3}\,\mathrm{m/s}$
   (b) $-8.17 \times 10^{-3}\,\mathrm{m/s}$
   (c) $-5.83 \times 10^{-3}\,\mathrm{m/s}$
   (d) $-1.36 \times 10^{-3}\,\mathrm{m/s}$

3. **Terminal velocity of the beam.** Use the default parameters ($N = 21$, large sphere at the middle node only) and simulate for 50 s. What is the terminal velocity of the middle node?

   (a) $-9.53 \times 10^{-3}\,\mathrm{m/s}$
   (b) $-8.17 \times 10^{-3}\,\mathrm{m/s}$
   (c) $-6.81 \times 10^{-3}\,\mathrm{m/s}$
   (d) $-5.83 \times 10^{-3}\,\mathrm{m/s}$

4. **Position of the beam.** In the simulation of Question 3, what is the y-coordinate of the middle node at $t = 50\,\mathrm{s}$?

   (a) $-2.9\,\mathrm{m}$
   (b) $-0.41\,\mathrm{m}$
   (c) $-0.29\,\mathrm{m}$
   (d) $-0.029\,\mathrm{m}$

5. **Spatial discretization.** Repeat Question 3 with $N = 3$, 5, 11, and 21 (keep $R_{\mathrm{mid}} = 0.025\,\mathrm{m}$ and $R = \Delta L/10$, so the small spheres shrink as N grows). Which statement best describes the magnitude of the terminal velocity?

Copyright notice: Do not copy, distribute, republish, upload, post or transmit anything from the files hosted on the class website without written permission of the instructor

(a) It decreases slightly as N grows and converges to about $5.83 \times 10^{-3}\,\mathrm{m/s}$.

(b) It is exactly the same for every N, because the elastic forces are internal and cancel out.

(c) It increases steadily with N and does not converge to a fixed value.

(d) It decreases by more than a factor of 2 between $N = 3$ and $N = 21$.

Part 2: Simply supported beam

Unless a question says otherwise, use $N = 50$, $\Delta t = 10^{-2}\,\mathrm{s}$, total time 1 s, and the parameters of Part 2. "Deflection" means $|y_{\max}|$ at $t = 1\,\mathrm{s}$. Euler–Bernoulli (EB) predictions use Eq. (6) with $d = 0.75\,\mathrm{m}$.

6. **Small load.** For $P = 2000\,\mathrm{N}$, what is the simulated deflection $|y_{\max}|$?

   (a) 0.019 m

   (b) 0.037 m

   (c) 0.074 m

   (d) 0.37 m

7. **Location of the maximum deflection.** For $P = 2000\,\mathrm{N}$, at approximately what x-coordinate is the node with the largest downward deflection?

   (a) 0.25 m

   (b) 0.50 m (mid-span)

   (c) 0.56 m

   (d) 0.75 m (under the load)

8. **Steady state.** The model in Part 2 has no physical damping (no viscous force). For $P = 2000\,\mathrm{N}$, plot $y_{\max}$ versus time. Which statement best describes $y_{\max}(t)$?

   (a) It oscillates forever with a constant amplitude, because the model has no damping.

   (b) It becomes steady within a few time steps, because implicit Euler adds artificial (numerical) damping.

   (c) It becomes steady only near $t = 1\,\mathrm{s}$, because the material of the beam provides weak physical damping.

   (d) It oscillates with an amplitude that grows without bound as time goes on.

9. **Large load.** For $P = 20{,}000\,\mathrm{N}$, what is the simulated deflection $|y_{\max}|$?

   (a) 0.24 m

   (b) 0.38 m

   (c) 0.57 m

   (d) 0.76 m

Copyright notice: Do not copy, distribute, republish, upload, post or transmit anything from the files hosted on the class website without written permission of the instructor

10. **Where beam theory breaks down.** Define the relative difference

$$e(P) = \frac{y_{\max}^{\mathrm{EB}} - |y_{\max}^{\mathrm{sim}}|}{y_{\max}^{\mathrm{EB}}}.$$

Make a plot of $|y_{\max}|$ versus P for $20\,\mathrm{N} \le P \le 20{,}000\,\mathrm{N}$, using both the simulation and Eq. (6). Among the loads below, which is the *smallest* load at which $e(P)$ exceeds 10%?

(a) 1000 N

(b) 2000 N

(c) 4000 N

(d) 8000 N

11. **Why they differ.** At $P = 20{,}000\,\mathrm{N}$, the simulation predicts a much smaller deflection than Euler–Bernoulli theory. What is the main reason?

(a) Beam theory is linear: it assumes small slopes and a fixed, undeformed geometry; the simulation does not.

(b) The artificial damping of implicit Euler removes part of the simulated deflection at large loads.

(c) The stretching springs make the discrete beam much stiffer, because the beam stretches a lot at large loads.

(d) The simulation includes the mass (inertia) of the beam, while Euler–Bernoulli beam theory does not.

Tip: at the end of the $P = 20{,}000\,\mathrm{N}$ simulation, print the x-coordinate of the last node and the change in length of the stretching springs. They will help you with Question 11.

Copyright notice: Do not copy, distribute, republish, upload, post or transmit anything from the files hosted on the class website without written permission of the instructor

Appendix: Analytical Solutions

The derivations below are symbolic. Use them to check your simulation results; substituting the parameters of each question gives the expected answer.

A. Buoyant free fall ($\mu = 0$, equal radii)

With equal radii, every node has the same mass and weight, the beam stays straight, and the elastic forces are zero. Each node obeys

$$m_k \ddot{y}_k = -W_k \qquad \Longrightarrow \qquad \ddot{y}_k = -\frac{\frac{4}{3}\pi R^3(\rho_{\text{metal}} - \rho_{\text{fluid}})\, g}{\frac{4}{3}\pi R^3 \rho_{\text{metal}}} = -g\left(1 - \frac{\rho_{\text{fluid}}}{\rho_{\text{metal}}}\right). \tag{7}$$

The acceleration is smaller than g because of buoyancy, and it does not depend on R. Starting from rest, $\dot{y}_k(t) = \ddot{y}_k\, t$. For a constant acceleration, implicit Euler gives this velocity exactly at every time step. (The model ignores the "added mass" of the fluid that moves with the sphere, so the inertia is m_k only.)

B. Terminal velocity of one sphere (Stokes drag)

With drag, the equation of motion of a single sphere (or of every node of a beam with equal radii) is

$$m\dot{v} = -W - 6\pi\mu R\, v. \tag{8}$$

Starting from rest,

$$v(t) = -v_\infty\left(1 - e^{-t/\tau}\right), \qquad v_\infty = \frac{W}{6\pi\mu R} = \frac{2R^2(\rho_{\text{metal}} - \rho_{\text{fluid}})\, g}{9\mu}, \qquad \tau = \frac{m}{6\pi\mu R} = \frac{2R^2\rho_{\text{metal}}}{9\mu}. \tag{9}$$

This is Stokes' law. For the parameters in this homework, τ is much smaller than Δt, so the sphere reaches v_∞ almost at once. Implicit Euler handles this without difficulty. Each step multiplies the difference $v - (-v_\infty)$ by $\tau/(\tau + \Delta t) < 1$, and at steady state the discrete equation is satisfied exactly by $v = -v_\infty$.

C. Terminal velocity of the whole beam (unequal radii)

Elastic forces are internal. The elastic energy depends only on the *shape* of the beam (lengths and turning angles). It does not change if the whole beam is translated. Therefore

$$\sum_{k=1}^{N} \frac{\partial E^{\text{elastic}}}{\partial y_k} = 0 \qquad \text{and} \qquad \sum_{k=1}^{N} \frac{\partial E^{\text{elastic}}}{\partial x_k} = 0. \tag{10}$$

Adding the y-equations of motion of all nodes, the elastic forces cancel:

$$\sum_{k=1}^{N} m_k \ddot{y}_k = -\sum_{k=1}^{N} W_k - 6\pi\mu \sum_{k=1}^{N} R_k\, \dot{y}_k. \tag{11}$$

Copyright notice: Do not copy, distribute, republish, upload, post or transmit anything from the files hosted on the class website without written permission of the instructor

Steady state. Once the beam reaches its final (bent) shape, the shape no longer changes, so every node moves with the same velocity v and all accelerations are zero. Then

$$v_{\text{terminal}} = -\frac{\sum_k W_k}{6\pi\mu \sum_k R_k} = -\frac{2(\rho_{\text{metal}} - \rho_{\text{fluid}})\, g}{9\mu} \frac{\sum_k R_k^3}{\sum_k R_k}. \tag{12}$$

For equal radii this reduces to Stokes' law (Appendix B). The beam *shape* (and the bending stiffness) does not affect the terminal velocity; it only affects how the beam gets there.

Dependence on N. With R_{mid} at the middle node and $R = \Delta L/10 = L/\left(10(N-1)\right)$ at the other $N-1$ nodes,

$$\sum_k R_k = R_{\text{mid}} + \frac{L}{10}, \qquad \sum_k R_k^3 = R_{\text{mid}}^3 + \frac{L^3}{1000\,(N-1)^2}. \tag{13}$$

The sum of the radii does not depend on N, but the sum of the cubes does: the small spheres carry less weight as N grows. As $N \to \infty$, the small spheres carry no weight at all, but they still provide drag. Evaluate Eq. (12) for several N and for $N \to \infty$ to see the trend.

Position. After a short transient (while the beam bends into its final shape), the beam moves at a constant velocity, so $y_{\text{mid}}(t) \approx v_{\text{terminal}}\, t$ for large t. The transient adds only a small offset.

D. Euler–Bernoulli solution for a simply supported beam

Consider a simply supported beam of length l with a point load P at a distance a from the left support, and let $b = l - a$. Linear Euler–Bernoulli theory gives the deflection (positive downward) on the left of the load, $0 \le x \le a$:

$$w(x) = \frac{P\, b\, x}{6EIl}\left(l^2 - b^2 - x^2\right). \tag{14}$$

If $a > b$ (the load is closer to the right support), the maximum deflection is on the left of the load. Setting $w'(x) = 0$ gives

$$x_{\max} = \sqrt{\frac{l^2 - b^2}{3}}, \qquad w_{\max} = w(x_{\max}) = \frac{P\, b\, \left(l^2 - b^2\right)^{1.5}}{9\sqrt{3}\, EIl}. \tag{15}$$

In general b is replaced by $c = \min(a, b)$, which gives Eq. (6). Two properties are useful:

- The maximum deflection is *not* under the load and *not* at mid-span. It lies between the two.
- The deflection is *linear* in P: doubling the load doubles the deflection, at any load.

Discretization note. With $N = 50$, the load is applied at $x = 37/49 = 0.7551\,\text{m}$, not at $0.75\,\text{m}$. Even at very small loads, this gives a difference of about 1.6% between the simulation and Eq. (6) with $d = 0.75\,\text{m}$. If you evaluate Eq. (6) with $d = 0.7551\,\text{m}$ instead, the agreement at small loads is much better. Either choice gives the same answers to the questions.

E. Assumptions of Euler–Bernoulli theory

The formula in Appendix D rests on three simplifications:

Copyright notice: Do not copy, distribute, republish, upload, post or transmit anything from the files hosted on the class website without written permission of the instructor

1. **Small slopes.** The curvature is approximated by $w''(x)$. The exact curvature is $w''/(1 + w'^2)^{3/2}$.

2. **Undeformed geometry.** Equilibrium is written on the straight, undeformed beam. The load and the reactions are assumed to keep their original moment arms.

3. **Fixed span.** The horizontal distance between the supports stays l, so the roller does not move.

These make the theory linear, and they are accurate when the deflection is small compared with l. The discrete simulation makes none of them. The discrete bending energy uses the exact turning angle, all forces act on the deformed shape, and the roller node is free to slide in x. Axial stretching is included, but for a slender beam it is very small: compare EA with EI/l^2.

F. Why the undamped beam reaches a steady state

The beam in Part 2 has no physical damping, so the exact solution of the dynamic problem oscillates forever about the static equilibrium. Homework 1 (Appendix D) showed that implicit Euler multiplies the amplitude of an oscillation with angular frequency ω by

$$\left(1 + \omega^2 \Delta t^2\right)^{-1/2} \tag{16}$$

at every step. The lowest (fundamental) bending frequency of a simply supported beam is

$$\omega_1 = \left(\frac{\pi}{l}\right)^2 \sqrt{\frac{EI}{\rho A}}, \qquad A = \pi \left(R^2 - r^2\right). \tag{17}$$

Evaluate $\omega_1 \Delta t$ for this beam. When $\omega_1 \Delta t \gg 1$, the numerical damping removes almost all of the oscillation in every step, so the simulation settles to the static solution almost at once. In this sense, the dynamic simulation acts as a solver for the static problem. The static equilibrium does not depend on the damping, so the final deflection is not affected by it.

Copyright notice: Do not copy, distribute, republish, upload, post or transmit anything from the files hosted on the class website without written permission of the instructor